

Chapter 4: Data Transfers, Addressing, and Arithmetic

4.1. Data Transfer Instructions

These move data between registers, memory, and I/O ports.

They **do not** affect flags (except XLAT and a few string instructions).

1.1 Basic Data Transfer

Data transfer instructions are used to move data from one location to another. These instructions copy the data; they do not change the original source.

1. MOV (Move)

Moves data from source to destination.

Examples:

MOV AX, BX

MOV AL, 25H

MOV [2000H], AX

2. MVI / MOV Immediate (Load Immediate Data)

Loads a constant value directly into a register.

Examples:

MOV CL, 0AH

MOV AX, 1234H

4. XCHG (Exchange)

Exchanges the contents of two registers.

Example:

XCHG AX, BX

5. LEA (Load Effective Address)

Loads the address of a memory variable.

Example:

LEA SI, ARRAY

6. PUSH (Push to Stack)

Stores register contents onto the stack.

Examples:

PUSH AX

PUSH BX

7. POP (Pop from Stack)

Retrieves data from the stack.

Examples:

POP DX

POP CX

8. IN (Input from Port)

Reads data from an input port.

Example:

IN AL, 20H

9. OUT (Output to Port)

Sends data from a register to an output port.

Example:

OUT 30H, AL

Complete Example Program:

MOV AX, 1234H

MOV BX, AX

XCHG AX, BX

MOV [2000H], AX

LEA SI, 2000H

PUSH AX

POP DX

Instruction	Description	Example
MOV dest, src	Copies data from src to dest	MOV AX, BX
XCHG dest, src	Exchanges data between operands	XCHG AX, BX
LEA reg, mem	Loads the effective address into a register	LEA BX, [ARRAY]
PUSH src	Pushes data onto the stack	PUSH AX
POP dest	Pops data from the stack	POP BX

Rules:

- You can't MOV directly from memory to memory.
- MOV between different sizes (byte/word/dword) must match.

2. Addressing Modes

Addressing modes specify **where operands are located** (register, memory, or immediate data).

Mode	Example	Description	Detailed Notes
Immediate	MOV AL, 5	Operand is a constant value embedded in the instruction.	- The value 5 is part of the instruction itself. - No memory fetch is needed; the CPU loads the value directly into the register. - Used for initializing registers or constants. - Fastest addressing mode.
Register	MOV AX, BX	Data is stored in a register; instruction moves data between registers.	- Both source and destination are CPU registers. - No memory access; operation is entirely internal. - Fast and efficient. - Commonly used for arithmetic, logic, and temporary storage.
Direct	MOV AL, [1234H]	Memory address is specified directly in the instruction.	- [1234H] is the exact memory address where the data is stored. - CPU fetches data from that memory location. - Useful for accessing global/static variables. - Slower than register or immediate because memory access takes time.
Register Indirect	MOV AL, [BX]	Register contains the memory address; CPU fetches data from that address.	- The value in register BX is treated as a pointer to memory. - Allows flexible access to arrays or buffers. - Supports base register changes for dynamic memory addressing. - Faster than direct memory addressing if used with efficient registers.
Base + Index	MOV AL, [BX+SI]	Combines two registers (base + index) to compute the memory address.	- BX is the base register, SI is the index register. - Effective address = Base + Index. - Used for arrays, structures, and tables. - Can also be combined with a displacement to access a specific element (MOV AL, [BX+SI+05H]). - Highly flexible for memory-

			intensive programs.
--	--	--	---------------------

3. Arithmetic Instructions

These perform addition, subtraction, increment, decrement, and related operations.

3.1 Basic Arithmetic

Instruction	Description	Example
ADD dest, src	$\text{dest} \leftarrow \text{dest} + \text{src}$	ADD AX, BX
ADC dest, src	$\text{dest} \leftarrow \text{dest} + \text{src} + \text{CF}$	Used for multi-byte addition
SUB dest, src	$\text{dest} \leftarrow \text{dest} - \text{src}$	SUB AL, 1
SBB dest, src	$\text{dest} \leftarrow \text{dest} - \text{src} - \text{CF}$	Used for multi-byte subtraction
INC dest	$\text{dest} \leftarrow \text{dest} + 1$	INC AX
DEC dest	$\text{dest} \leftarrow \text{dest} - 1$	DEC BX
NEG dest	$\text{dest} \leftarrow 0 - \text{dest}$	Two's complement negate

Flags affected: CF, ZF, SF, OF,

Flag	Full Name	Purpose / Description	When It Is Set
CF	Carry Flag	Indicates a carry out of the most significant bit (MSB) in arithmetic operations.	- Set if an addition produces a carry beyond the register size. - Set if a subtraction requires a borrow. Example: ADD AL, 250 (AL=10 → result 260) sets CF.
ZF	Zero Flag	Indicates whether the result of an operation is zero.	- Set if the result of arithmetic or logic operation = 0. - Cleared if result ≠ 0. Example: SUB AL, AL sets ZF.
SF	Sign Flag	Indicates the sign of the result.	- Set if the result is negative (MSB = 1). - Cleared if the result is positive (MSB = 0). Example: SUB AL, 20 when AL = 10 sets SF.
OF	Overflow Flag	Indicates signed arithmetic overflow.	- Set if signed result is too large or too small for the destination register. - Example: ADD AL, 127 when AL = 1 → result = -128 (overflow) sets OF.

3.2 Multiplication and Division

Instruction	Operation	Example
MUL src	Unsigned multiply	MUL BL $\rightarrow$ AX = AL $\times$ BL
IMUL src	Signed multiply	IMUL BX
DIV src	Unsigned divide	DIV BL
IDIV src	Signed divide	IDIV BX

4. Example Program

DATA SEGMENT

NUM1 DB 10

NUM2 DB 20

SUM DB ?

DATA ENDS

CODE SEGMENT

MOV AX, DATA

MOV DS, AX

MOV AL, NUM1

ADD AL, NUM2

MOV SUM, AL

MOV AH, 4CH

INT 21H

CODE ENDS

END

Explanation:

- Moves data from memory to registers (MOV)
- Adds two numbers (ADD)
- Stores the result in memory
- Terminates program (INT 21H)

4.2 Addition and Subtraction

Used for **arithmetic operations** between registers, immediate data, or memory.

Addition Instructions

Instruction	Operation	Example
ADD dest, src	$\text{dest} \leftarrow \text{dest} + \text{src}$	ADD AX, BX
ADC dest, src	$\text{dest} \leftarrow \text{dest} + \text{src} + \text{CF}$	For multi-byte addition

Subtraction Instructions

Instruction	Operation	Example
SUB dest, src	$\text{dest} \leftarrow \text{dest} - \text{src}$	SUB AX, 10
SBB dest, src	$\text{dest} \leftarrow \text{dest} - \text{src} - \text{CF}$	For multi-byte subtraction

Increment / Decrement

Instruction	Operation	Example
INC dest	$\text{dest} \leftarrow \text{dest} + 1$	INC BX
DEC dest	$\text{dest} \leftarrow \text{dest} - 1$	DEC CX

□ Example:

MOV AL, 5

ADD AL, 3 ; AL = 8

SUB AL, 2 ; AL = 6

4.3 Data-Related Operators and Directives

These are **assembler directives** used to define, allocate, or label data.

Directive	Meaning	Example
DB	Define Byte	VAL1 DB 10
DW	Define Word (2 bytes)	VAL2 DW 2000
DD	Define Double Word (4 bytes)	NUM DD 12345678h
DQ	Define Quad Word (8 bytes)	QNUM DQ 100000000h
EQU	Define Constant	PI EQU 3.14
DUP	Duplicate Initialization	ARRAY DB 5 DUP(0)
OFFSET	Address of variable	MOV BX, OFFSET VAR1

Directives tell the assembler **how to store data**, not instructions for the CPU.

🔄 4.5 JMP and LOOP Instructions

These control **program flow**, enabling iteration and conditional branching.

JMP (Unconditional Jump)

JMP causes the processor to **unconditionally transfer control** to another location in the program.

Execution continues from the specified label (address).

◆ Syntax:

JMP label

◆ Description:

- It changes the **Instruction Pointer (IP)** (or **EIP/RIP** in 32/64-bit).
- The **jump is unconditional** — it always happens.
- Commonly used for:
 - Repeating code sections
 - Skipping over code blocks
 - Implementing control structures (loops, conditionals)

Instruction	Description	Example
JMP label	Jumps to label unconditionally.	JMP START

Conditional Jumps

Depend on flag settings after comparison or arithmetic operations.

Instruction	Meaning
JE / JZ	Jump if Equal / Zero
JNE / JNZ	Jump if Not Equal / Not Zero
JG / JA	Jump if Greater / Above
JL / JB	Jump if Less / Below

Example:

```
CMP AX, BX
JE EQUAL_LABEL
```

◆ *Example:*

```
MOV AX, 0
MOV BX, 5
```

START:

```
ADD AX, BX    ; AX = AX + BX
DEC BX        ; BX = BX - 1
CMP BX, 0     ; Compare BX with 0
JNZ START     ; Jump if Not Zero (loop back)
```

□ **Explanation:**

- JNZ (Jump if Not Zero) is a *conditional jump* based on flags.
- The program keeps jumping back to START until BX = 0.
- The unconditional version would use JMP START (always jump back, infinite loop).

LOOP Instruction

Uses the counter register (CX, ECX, or RCX).

Instruction	Description	Example
LOOP label	Decrements CX; jumps if not zero.	LOOP AGAIN

Example:

```
MOV CX, 5
AGAIN:
    ; do something
    LOOP AGAIN
```

This repeats the block **5 times**.

🔄 B. LOOP — Loop Instruction

◆ *Purpose:*

The LOOP instruction **decrements the counter register** (CX, ECX, or RCX) and **jumps to the label** if the counter is not zero.

◆ *Syntax:*

```
LOOP label
```


◆ *Description:*

- Automatically **decreases CX by 1**.
- If $CX \neq 0 \rightarrow$ **jump to label**.
- If $CX = 0 \rightarrow$ **exit the loop** (continue to next instruction).
- Commonly used for fixed iteration loops.

◆ *Example:*

MOV CX, 5 ; Initialize loop counter

MOV AX, 0 ; Clear accumulator

SUM_LOOP:

ADD AX, CX ; $AX = AX + CX$

LOOP SUM_LOOP ; Decrement CX and loop if not zero

□ **Explanation:**

- The loop runs 5 times.
- On each iteration:
 - CX is decremented.
 - AX accumulates the sum $5 + 4 + 3 + 2 + 1$.

After $CX = 0$, execution continues after the loop

Difference Between JMP and LOOP

Feature	JMP	LOOP
Type	Unconditional / Conditional jump	Controlled by CX/ECX counter
Counter Used	None	Uses CX, ECX, or RCX
Automatic Decrement	No	Yes, decrements counter
Usage	Any kind of jump	Fixed iteration loops
Example	JMP LABEL	LOOP LABEL

Combined Example

MOV CX, 3

MOV AX, 0

START:

ADD AX, 2 ; Add 2 each time

LOOP START ; Repeat 3 times

```
JMP END_PROGRAM    ; Unconditional jump to end
```

```
END_PROGRAM:
```

```
    HLT            ; Stop execution
```

```
.
```